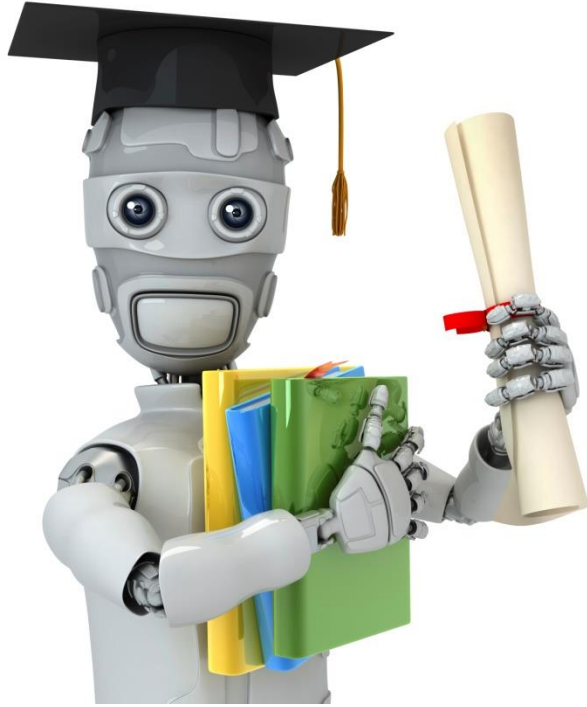




Machine Learning

Linear Regression with multiple variables

Multiple features

Multiple features (variables).

Size (feet ²)	Price (\$1000)
$\rightarrow x$	$y \leftarrow$
2104	460
1416	232
1534	315
852	178
...	...

$$\underline{h_{\theta}(x) = \theta_0 + \theta_1 x}$$

Multiple features (variables).

Size (feet ²)	Number of bedrooms	Number of floors	Age of home (years)	Price (\$1000)
2104	5	1	45	460
1416	3	2	40	232
1534	3	2	30	315
852	2	1	36	178
...	...	...	...	...

Notation:

n = number of features

$x^{(i)}$ = input (features) of i^{th} training example.

$x_j^{(i)}$ = value of feature j in i^{th} training example.

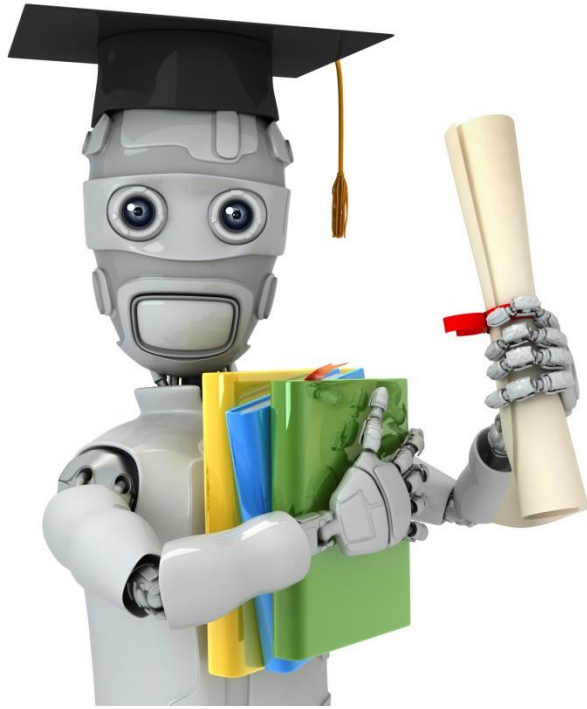
Hypothesis:

Previously: $h_{\theta}(x) = \theta_0 + \theta_1 x$

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

For convenience of notation, define $x_0 = 1$.

Multivariate linear regression.



Machine Learning

Linear Regression with multiple variables

Gradient descent for multiple variables

Hypothesis: $h_{\theta}(x) = \theta^T x = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$

Parameters: $\theta_0, \theta_1, \dots, \theta_n$

Cost function:

$$J(\theta_0, \theta_1, \dots, \theta_n) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Gradient descent:

Repeat {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \dots, \theta_n)$$

} (simultaneously update for every $j = 0, \dots, n$)

Gradient Descent

Previously ($n=1$):

Repeat {

$$\theta_0 := \theta_0 - \underbrace{\alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})}_{\frac{\partial}{\partial \theta_0} J(\theta)}$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})x^{(i)}$$

(simultaneously update θ_0, θ_1)

}

New algorithm ($n \geq 1$):

Repeat {

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})x_j^{(i)}$$

(simultaneously update θ_j for
 $j = 0, \dots, n$)

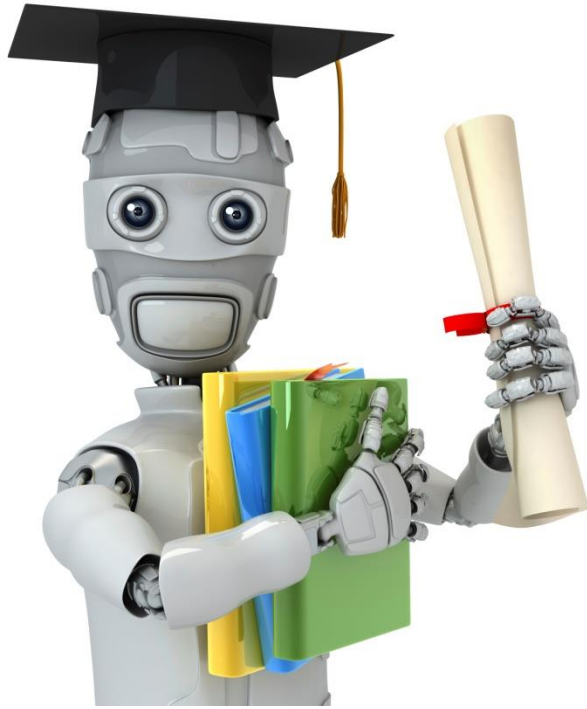
}

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})x_0^{(i)}$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})x_1^{(i)}$$

$$\theta_2 := \theta_2 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})x_2^{(i)}$$

...



Machine Learning

Linear Regression with multiple variables

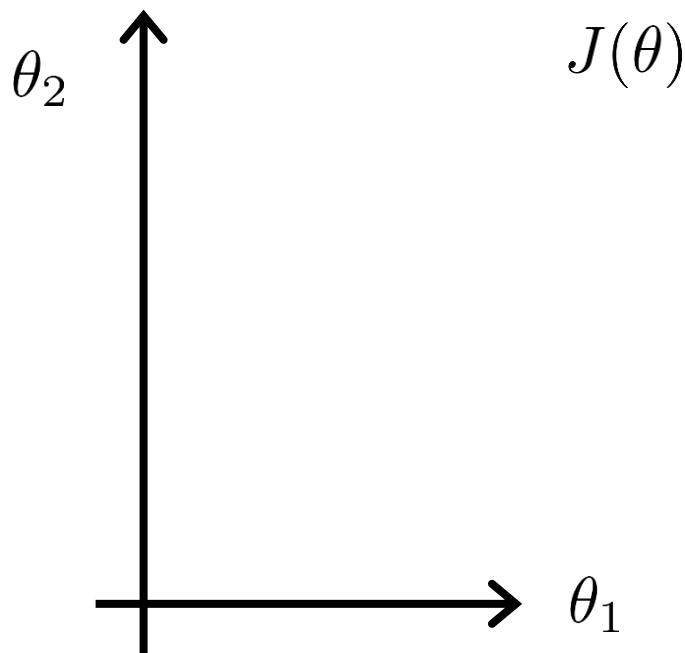
Gradient descent in practice I: Feature Scaling

Feature Scaling

Idea: Make sure features are on a similar scale.

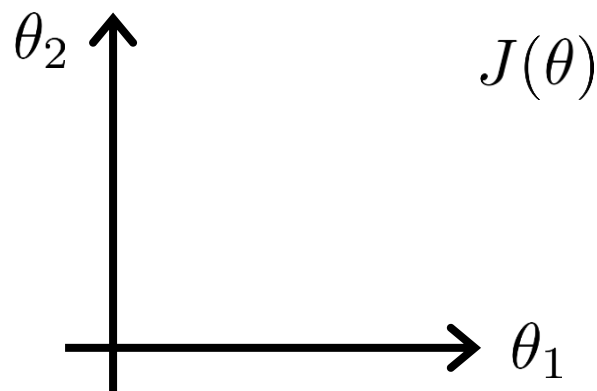
E.g. x_1 = size (0-2000 feet²)

x_2 = number of bedrooms (1-5)



$$x_1 = \frac{\text{size (feet}^2\text{)}}{2000}$$

$$x_2 = \frac{\text{number of bedrooms}}{5}$$



Feature Scaling

Get every feature into approximately a $-1 \leq x_i \leq 1$ range.

$$x_0 = 1$$

$$0 \leq x_1 \leq 3 \quad \checkmark$$

$$-2 \leq x_2 \leq 0.5 \quad \checkmark$$

$$-100 \leq x_3 \leq 100 \quad \times$$

$$-0.0001 \leq x_4 \leq 0.0001 \quad \times$$

$$-3 \text{ to } 3 \quad \checkmark$$

$$-\frac{1}{3} \text{ to } \frac{1}{3} \quad \checkmark$$

Mean normalization

Replace x_i with $x_i - \mu_i$ to make features have approximately zero mean
(Do not apply to $x_0 = 1$).

E.g. $x_1 = \frac{\text{size} - 1000}{2000}$

$$x_2 = \frac{\#bedrooms - 2}{5}$$

$$-0.5 \leq x_1 \leq 0.5, -0.5 \leq x_2 \leq 0.5$$

Handwritten diagram illustrating the normalization process:

For x_1 :

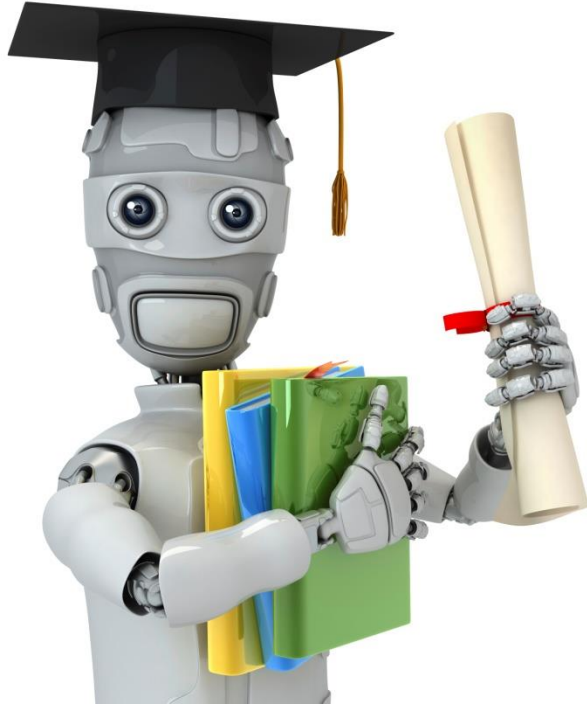
$$x_1 \leftarrow \frac{x_1 - \mu_1}{s_1}$$

Annotations for x_1 :

- μ_1 is circled in red, with an arrow pointing to it from the text "avg value of x_1 in training set".
- s_1 is circled in red, with an arrow pointing to it from the text "range (max-min) (or standard deviation)".

For x_2 :

$$x_2 \leftarrow \frac{x_2 - \mu_2}{s_2}$$



Machine Learning

Linear Regression with multiple variables

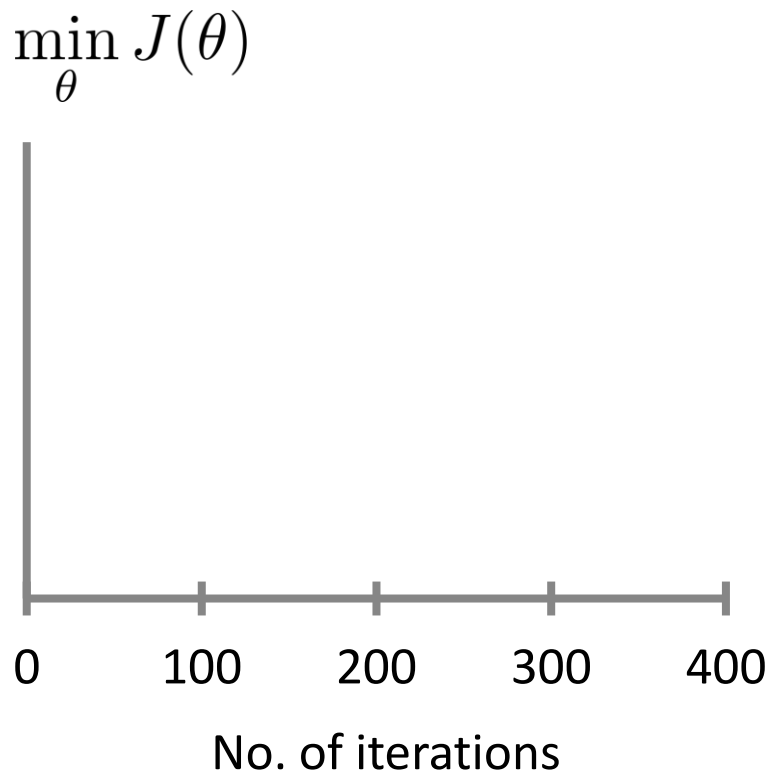
Gradient descent in practice II: Learning rate

Gradient descent

$$\rightarrow \theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

- “Debugging”: How to make sure gradient descent is working correctly.
- How to choose learning rate α .

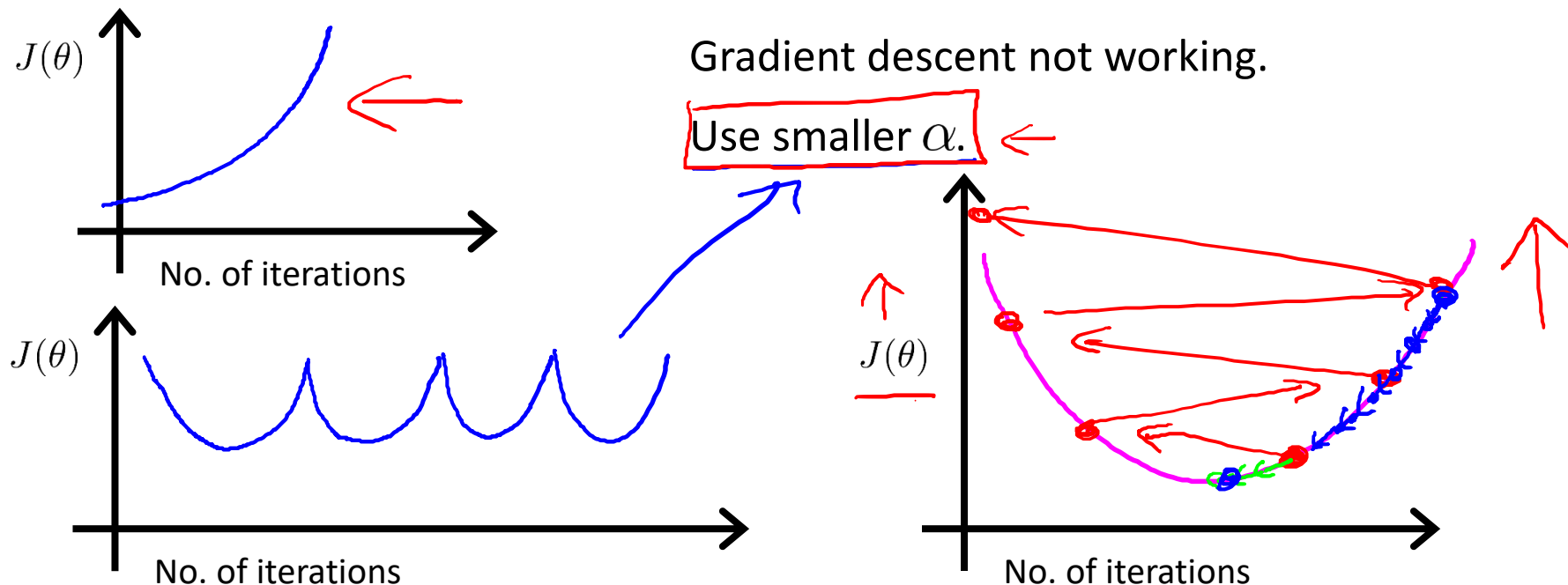
Making sure gradient descent is working correctly.



Example automatic
convergence test:

Declare convergence if $J(\theta)$
decreases by less than 10^{-3}
in one iteration.

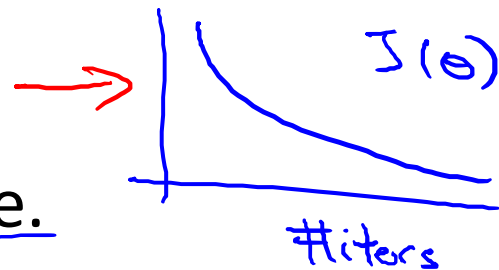
Making sure gradient descent is working correctly.



- For sufficiently small α , $J(\theta)$ should decrease on every iteration.
- But if α is too small, gradient descent can be slow to converge.

Summary:

- If α is too small: slow convergence.
- If α is too large: $J(\theta)$ may not decrease on every iteration; may not converge.

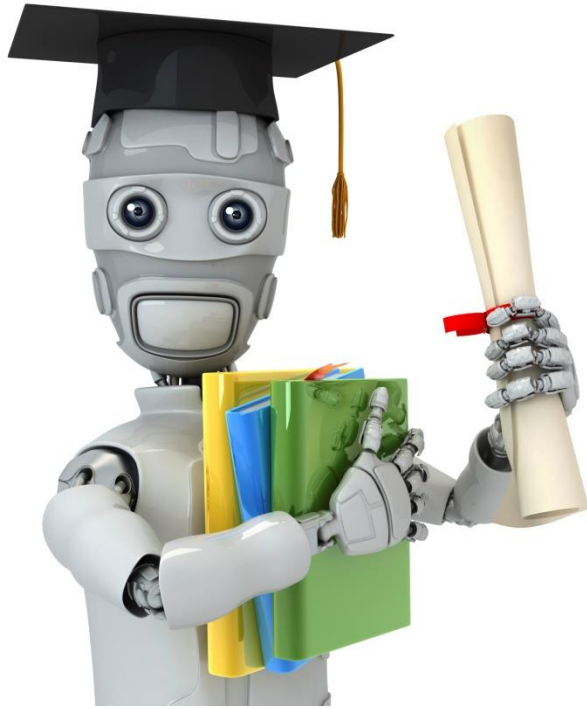


(Slow converge also possible.)

To choose α , try

$\dots, \underline{0.001}, \underline{0.003}, \underline{0.01}, \underline{0.03}, \underline{0.1}, \underline{0.3}, \underline{1}, \dots$

Arrows indicate the sequence of values: $\uparrow$ (under 0.001), $\nearrow$ (from 0.001 to 0.003, labeled $3\times$), $\nearrow$ (from 0.003 to 0.01, labeled $\approx 3\times$), $\nearrow$ (from 0.01 to 0.03, labeled $3\times$), $\nearrow$ (from 0.03 to 0.1, labeled $\approx 3\times$), $\uparrow$ (under 0.1), $\uparrow$ (under 0.3), $\uparrow$ (under 1).



Machine Learning

Linear Regression with multiple variables

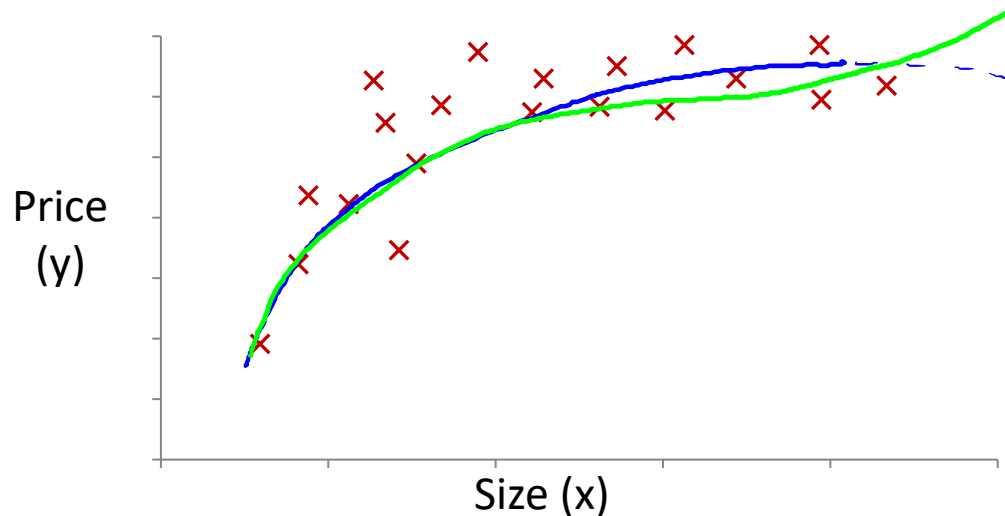
Features and
polynomial regression

Housing prices prediction

$$h_{\theta}(x) = \theta_0 + \theta_1 \times \textit{frontage} + \theta_2 \times \textit{depth}$$



Polynomial regression



$$\rightarrow \theta_0 + \theta_1 x + \theta_2 x^2$$

$$\rightarrow \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3$$

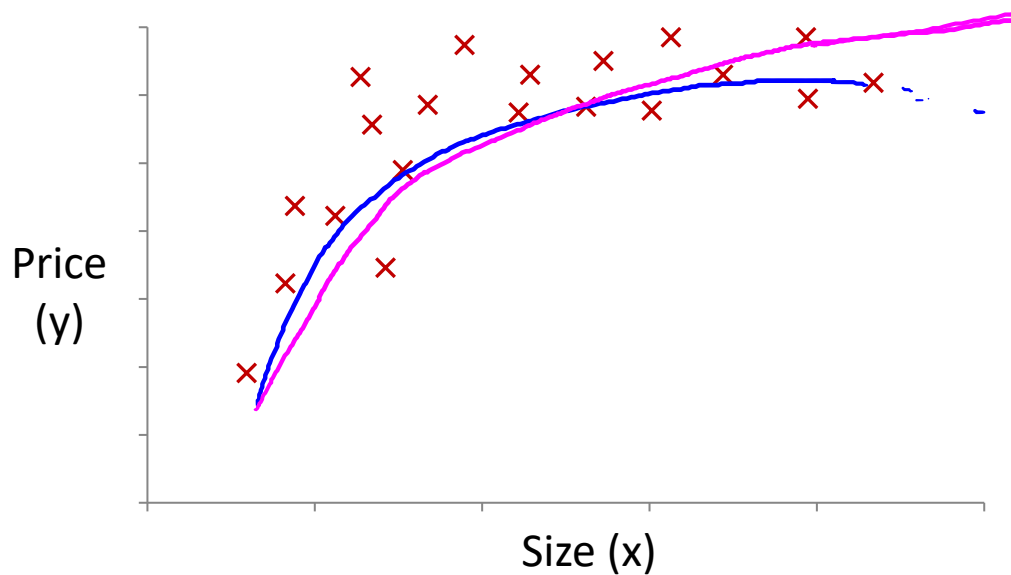
$$\begin{aligned} h_{\theta}(x) &= \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 \\ &= \theta_0 + \theta_1(\text{size}) + \theta_2(\text{size})^2 + \theta_3(\text{size})^3 \end{aligned}$$

$$x_1 = (\text{size})$$

$$x_2 = (\text{size})^2$$

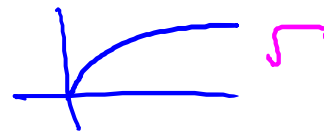
$$x_3 = (\text{size})^3$$

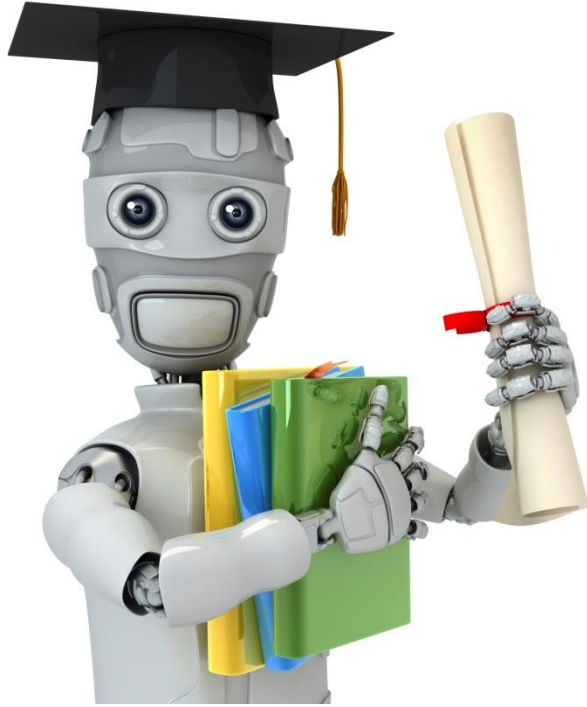
Choice of features



$$\rightarrow h_{\theta}(x) = \theta_0 + \theta_1(\text{size}) + \theta_2(\text{size})^2$$

$$\rightarrow h_{\theta}(x) = \theta_0 + \theta_1(\text{size}) + \theta_2\sqrt{(\text{size})}$$



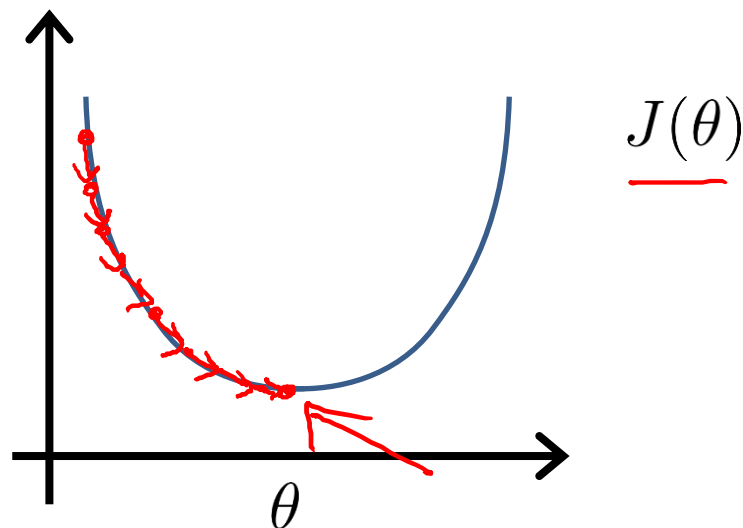


Machine Learning

Linear Regression with multiple variables

Normal equation

Gradient Descent



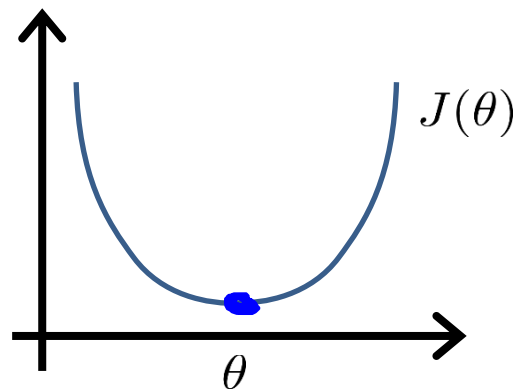
Normal equation: Method to solve for θ
analytically.

Intuition: If 1D ($\theta \in \mathbb{R}$)

→

$\frac{d}{d\theta} J(\theta) = \dots \stackrel{\text{set}}{=} 0$

Solve for θ



$$\theta \in \mathbb{R}^{n+1} \quad \underline{J(\theta_0, \theta_1, \dots, \theta_m)} = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

$$\underline{\frac{\partial}{\partial \theta_j} J(\theta)} = \dots \stackrel{\text{set}}{=} 0 \quad (\text{for every } j)$$

Solve for $\theta_0, \theta_1, \dots, \theta_n$

Examples: $m = 4$.

	Size (feet ²)	Number of bedrooms	Number of floors	Age of home (years)	Price (\$1000)
x_0	x_1	x_2	x_3	x_4	y
1	2104	5	1	45	460
1	1416	3	2	40	232
1	1534	3	2	30	315
1	852	2	1	36	178

$X = \begin{bmatrix} 1 & 2104 & 5 & 1 & 45 \\ 1 & 1416 & 3 & 2 & 40 \\ 1 & 1534 & 3 & 2 & 30 \\ 1 & 852 & 2 & 1 & 36 \end{bmatrix}$
 $m \times (n+1)$

$y = \begin{bmatrix} 460 \\ 232 \\ 315 \\ 178 \end{bmatrix}$
 m -dimensional vector

$\theta = (X^T X)^{-1} X^T y$

m examples $(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})$; n features.

$$\underline{x^{(i)}} = \begin{bmatrix} x_0^{(i)} \\ x_1^{(i)} \\ x_2^{(i)} \\ \vdots \\ x_n^{(i)} \end{bmatrix} \in \mathbb{R}^{n+1}$$

X
(design matrix)

$$= \begin{bmatrix} \text{---} (x^{(1)})^T \text{---} \\ \text{---} (x^{(2)})^T \text{---} \\ \vdots \\ \text{---} (x^{(m)})^T \text{---} \end{bmatrix}$$

E.g. If $\underline{x^{(i)}} = \begin{bmatrix} 1 \\ x_1^{(i)} \end{bmatrix}$

$$\Theta = (X^T X)^{-1} X^T y$$

$$\begin{matrix} m \times (n+1) \\ \begin{bmatrix} 1 & x_1^{(1)} \\ 1 & x_1^{(2)} \\ \vdots & \vdots \\ 1 & x_1^{(m)} \end{bmatrix} \end{matrix} \quad \begin{matrix} m \times 2 \\ \text{---} \end{matrix} \quad \begin{matrix} \text{---} \\ y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix} \end{matrix}$$

$$\theta = (X^T X)^{-1} X^T y$$



$(X^T X)^{-1}$ is inverse of matrix $X^T X$.

Set $A = X^T X$

$$(X^T X)^{-1} = A^{-1}$$

Octave: `pinv(X' * X) * X' * y`

$$\text{pinv}(X^T * X) * X^T * y$$

$$\theta = (X^T X)^{-1} X^T y$$

$\min_{\theta} J(\theta)$

X'

X^T

~~Feature Scaling~~

$$0 \leq x_1 \leq 1$$

$$0 \leq x_2 \leq 1000$$

$$0 \leq x_3 \leq 10^{-5}$$



m training examples, n features.

Gradient Descent

- • Need to choose α .
- • Needs many iterations.
- Works well even when n is large.



$$\underline{n = 10^6}$$



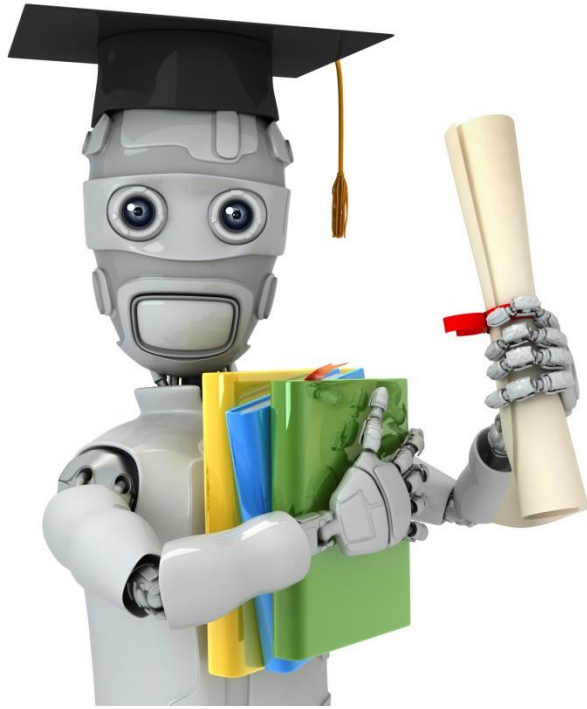
Normal Equation

- • No need to choose α .
- • Don't need to iterate.
- Need to compute
- • $(X^T X)^{-1}$ $\frac{n \times n}{\quad}$ $\underline{O(n^3)}$
- Slow if n is very large.

$$n = 100$$

$$n = 1000$$

$$- - - \underline{n = 10000}$$



Machine Learning

Linear Regression with multiple variables

Normal equation
and non-invertibility
(optional)

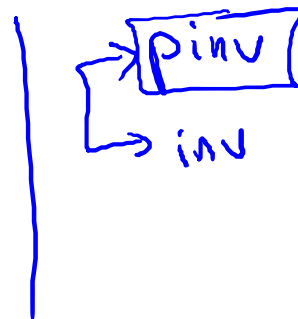
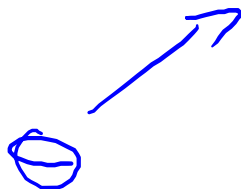
Normal equation

$$\theta = \underline{(X^T X)^{-1} X^T y}$$

$$\underline{X^T X}$$

- What if $\boxed{X^T X}$ is non-invertible? (singular/
degenerate)

- Octave: `pinv(X' * X) * X' * y`



What if $X^T X$ is non-invertible?

- Redundant features (linearly dependent).

E.g. $\begin{cases} \underline{x_1} = \text{size in feet}^2 \\ \underline{x_2} = \text{size in m}^2 \\ \underline{x_1 = (3.28)^2 x_2} \end{cases}$

$1\text{m} = 3.28\text{ feet}$

$\rightarrow m = 10 \leftarrow$

$\rightarrow n = 100 \leftarrow$

$\odot \in \mathbb{R}^{101}$

- Too many features (e.g. $m \leq n$).

- Delete some features, or use regularization.

$\downarrow$ later

